

Week-1 Report

Adaptive Learning, Diagnostic Assessment and Personalized Path-Planning Engine

Group 15

Student: Vuppula Hemanth Reddy

Roll Number: AI23BTECH11033

Github Repo: <https://github.com/vhemanth06/AdaptEngine>

1 Problem Boundary

This system is architected as a backend policy and recommendation engine specifically designed for quantitative disciplines such as mathematics, computer science etc. It continually assesses the learner's estimated mastery and time-bound constraints to construct optimized learning paths, recommend specific activities, and generate transparent, machine-readable justifications for all scheduling decisions.

Actors: Learner (the only external actor in scope) and the Engine.

Interfaces:

- (a) A Python library/API exposing diagnostic, planning, mastery-update, replanning, and explanation functions.
- (b) A thin CLI (and, if needed, a minimal Streamlit application) that calls this API.

In Scope: The following features and milestones are strictly within the scope of this prototype:

- Curriculum and learner-state modelling with mastery and uncertainty (M1).
- Bounded adaptive diagnostic compared against fixed/random baselines (M2).
- Prerequisite-valid, time-budgeted day-by-day planning for 5/10/15/30-minute budgets (M3).
- Explicit online mastery update and difficulty policy (M4).
- Spaced review and remediation scheduling (M5).
- Replanning on missed day, failed assessment, changed deadline, and changed daily budget (M6).
- Decision-trace-based explanations for every inclusion/omission/review/difficulty decision (M7).
- Comparative evaluation against static and greedy non-adaptive baselines (M8).

Out of Scope: To maintain a feasible engineering boundary, the following are explicitly excluded:

- A full learning-management system (LMS), enrolment, or grading/credentialing workflow.
- Production-grade authentication, multi-tenant UI, or mobile apps.
- Neural mastery models, reinforcement-learning planners, or an LLM explanation layer as core deliverables (all are considered optional extensions).

2 Assumptions

To define a clear boundary for the engine’s logic and our evaluation metrics, the project relies on the following structural assumptions:

- **Concept dependencies:** The curriculum is modeled as a Directed Acyclic Graph (DAG). We assume the prerequisite relationships defined in this graph are entirely accurate and must be treated as absolute hard constraints by the planning algorithm.
- **Lesson duration:** We assume that every learning activity takes a fixed, indivisible amount of time to complete (i.e., a learner cannot pause a 10-minute lesson halfway through). These baseline durations are either derived from median completion times in real datasets or provided as static estimates.
- **Item tagging:** Each exercise or diagnostic question is mapped to exactly one primary concept. We are excluding multi-skill tagging for this prototype to ensure that every correct or incorrect answer cleanly updates the mastery estimate for one specific topic without mathematical ambiguity.
- **Question difficulty:** Question difficulty is represented as a normalized continuous value between 0.05 and 0.95. This standard scale allows the engine to easily map and compare the difficulty of an item directly against a learner’s estimated mastery probability.
- **Learner behavior:** We assume learners are strictly bound by a fixed daily time budget and a final target deadline. Partial daily completion is not modeled; a missed study session is treated as exactly zero minutes of activity. Missing a day directly triggers the memory decay model and forces the engine to recalculate the learning path.
- **Mastery observability:** True mastery is a hidden latent state that we can only estimate through test performance. We assume all learner responses can be graded as strict binary outcomes (correct/incorrect), and that this pass/fail signal is sufficient to update our belief about their knowledge state.
- **Simulator validity:** Because static public datasets do not let us interactively test constraints like daily time limits or skipped days, our interactive evaluation relies on a synthetic learner simulator. We assume this simulator accurately mimics real human learning dynamics because its underlying parameters are sampled directly from real-world educational datasets.

3 Literature and System Landscape

To justify the engine’s underlying architecture, four representative approaches were compared across the core domains of mastery estimation, adaptive diagnosis, path planning, and spaced review.

The following table explicitly defines what concepts were adopted into this Minimum Viable Prototype (MVP) and what was deliberately excluded to ensure the system remains interpretable and mathematically constrained.

Domain	Core Idea	Adopted	not adopted (Why)
Mastery Estimation: Deep Knowledge Tracing vs. Bayesian Beta Distribution	DKT uses neural sequence models (RNNs) over exercise logs to predict future correctness, offering very high raw accuracy.	A Bayesian Beta Distribution model will be implemented to provide explicit, statistically interpretable variance (uncertainty) alongside expected mastery.	DKT is rejected for the core engine because it is an uninterpretable black box. Its latent state cannot generate the explicit mathematical triggers required for the system's decision-trace explanations.
Adaptive Diagnosis: Computerized Adaptive Testing (CAT) vs. Fixed-Order Tests	CAT uses item selection rules (like variance reduction) and explicit stopping rules to rapidly estimate a learner's ability without overtesting.	An uncertainty reduction selection rule will be implemented. The diagnostic will dynamically pick question difficulties to shrink uncertainty about the learner's state, halting as soon as an explicit stopping rule (like an uncertainty threshold or item cap) is met.	Complex features like full exposure control and multi dimensional item calibration will be excluded to keep the MVP manageable. Standard fixed order tests are also rejected for the core engine, but will be retained strictly as static baselines for later evaluation.
Path Planning: Set-to-Sequence Neural Networks vs. Constrained A* Search	Neural models rank candidate concepts into a study path based on the learner's state and the curriculum graph.	A "concept-aware" scoring approach will be adopted. Candidate concepts will be evaluated and scored against the learner's current state to figure out the most useful next step.	The neural network itself is excluded because the system requires explicit, traceable decisions. Constrained A* will be used instead, as neural models cannot mathematically guarantee zero prerequisite violations under a rigid time budget.
Spaced Review: Half-Life Regression (HLR)	HLR models how a learner's memory decays over time by calculating a specific "half-life" for recall probability based on their practice history.	HLR will be adopted directly as the review urgency model. It will calculate the forgetting risk, allowing review tasks to be mathematically scored so they can compete fairly against new learning tasks for the student's daily time budget.	The original paper's full, per-user feature regression will be omitted. To keep the MVP simple, standard literature default features will be used for the half-life parameters instead of training a complex model that requires massive amounts of historical data.

3.1 References

- Piech, C., et al. (2015). Deep Knowledge Tracing. *arXiv:1506.05908*.
- A Survey of Computerized Adaptive Testing (2024). *arXiv:2404.00712*.
- Set-to-Sequence Ranking-based Concept-aware Learning Path Recommendation (2023). *arXiv:2306.04234*.
- Settles, B., & Meeder, B. (2016). A Trainable Spaced Repetition Model for Language Learning. *ACL P16-1174*.

4 Data and Simulator Decision

Two public educational datasets and one custom transparent simulator were evaluated against the exact fields the engine requires to function: a prerequisite structure, explicit lesson durations, mastery evidence, and reproducible learner profiles.

4.1 Comparison of Data Sources

Data Source	Available Fields / Strengths	Limitations	Final Decision
ASSISTments (2009–2010)	Contains real student item-response logs, skill tags, and attempt counts. Useful for validating if a mastery-update model can accurately predict real behavior.	Lacks a prerequisite graph, specific lesson duration fields, and explicit remediation/review labels.	Will be used strictly offline as an external sanity-check to validate the mastery model. Will not be used as the interactive data source.
Junyi Academy	Provides large-scale practice logs with an explicit, real-world prerequisite graph. Proves that this type of hierarchical graph structure is realistic.	Still lacks the explicit remediation, review, and specific duration fields that the planner requires.	Will be used strictly as a design reference to help construct a realistically shaped prerequisite DAG.
Transparent Simulator	Provides every field the planner needs (durations, prerequisites, and ground-truth mastery). Easily supports strict time budgets and skipped days.	Synthetic data only. Creates a risk of testing bias if the simulator is built using the exact same mathematical rules as the planner, which could artificially inflate the engine’s performance scores.	Selected as the primary, interactive data source for the planner and the full M1–M8 evaluation loop.

4.2 Primary Strategy Selection

Because public datasets lack the controlled edge cases needed for interactive testing (like mathematically impossible deadlines or explicitly missed study days), a hybrid strategy will be implemented.

The transparent synthetic simulator will serve as the primary environment for testing the engine. To prevent the testing bias mentioned above, the simulator’s parameters (such as learning rates, slip/guess chances, and forgetting curves) will be sampled directly from empirical distributions found in the public datasets. Additionally, mismatched simulator models will be utilized to ensure the engine is evaluated fairly against unexpected learning behaviors.

4.3 Handling Unavailable Fields

When setting up the simulator with public data, any missing information will be handled in the following ways:

- **Missing Baseline Durations:** If a lesson does not have an exact total time recorded, the baseline duration will be calculated by taking the median time students spent on a single attempt and multiplying it by the number of practice questions. If there is no usable timing data at all, placeholder times will be assigned and clearly marked in the code.
- **Unmapped Prerequisites:** The system will not force fake connections between datasets. If a skill from ASSISTments does not map cleanly onto the Junyi curriculum graph, it will be left out of the graph structure. It will strictly be used as a separate benchmark for testing how students answer questions, ensuring no unsupported prerequisite links are accidentally created.
- **Missing Review Histories:** The spaced-repetition math relies on knowing how fast a concept is forgotten. If a concept lacks enough student history to calculate a specific memory decay rate, it will simply be assigned a default median value based on the rest of the dataset. This ensures the scheduling algorithm does not crash or break when trying to process new concepts.

5 Curriculum and Learner-State Schema

To support the path-planning and diagnostic algorithms, the underlying data structures will be divided into static curriculum entities and a dynamic learner-state tracker.

5.1 Entity Definitions

The system will implement the following core schema to manage the graph and track student progress:

- **Concept Node:** The foundational topic within the curriculum graph.
 - `concept_id`: A unique identifier for the topic.
 - `prerequisites`: A list of `concept_id` strings representing mandatory prior topics that must be mastered first.
- **Activity (Lesson and Exercise):** The specific tasks assigned to the learner.
 - `activity_id`: A unique string identifier.

- **type:** A flag indicating if the item is instructional (**LESSON**) or a testable question (**EXERCISE**).
- **target_concept:** The single **concept_id** this activity is mapped to.
- **duration:** An integer representing the indivisible time block (in minutes) required to complete the activity.
- **difficulty:** A float between 0.05 and 0.95, assigned strictly to exercises for the adaptive diagnostic phase.
- **Learner State:** The dynamic profile that updates after every learner interaction.
 - **mastery:** A dictionary mapping each **concept_id** to a probability value, representing the system’s current estimate of the student’s knowledge.
 - **uncertainty:** A dictionary tracking the statistical variance (confidence) for each concept estimate, which will shrink as the system gathers more right/wrong answers.

5.2 Sample Branching Graph

The curriculum will be organized as a Directed Acyclic Graph (DAG) using the **prerequisites** field to ensure logical progression. Below is a sample branching graph demonstrating how a minimal computer science sequence is structured:



6 Formal Planning Specification

To ensure the path-planning algorithm behaves predictably, the scheduling logic will be explicitly separated into hard constraints, soft objectives, infeasibility conditions, and a defined output plan structure. This guarantees that the generated study paths are mathematically valid and practically useful for the learner.

6.1 Hard Constraints

Hard constraints are absolute rules that the Constrained A^* search algorithm cannot violate under any circumstances:

- **Prerequisite Adherence:** An activity cannot be scheduled unless all of its parent prerequisite concepts in the curriculum DAG have reached the target mastery threshold.
- **Daily Time Limits:** The total duration of all lessons and exercises scheduled for a single day must be less than or equal to the learner’s fixed daily time budget.

6.2 Soft Objectives

Soft objectives act as the scoring function (the heuristic) to rank valid candidate tasks when building the schedule:

- **Review Prioritization:** Concepts that the Half-Life Regression (HLR) model flags as at risk of being forgotten will receive a score boost, prioritizing spaced review within the daily budget.
- **Path Unlocking:** Concepts that serve as prerequisites for a large number of downstream topics will be prioritized early in the schedule to prevent learning bottlenecks.

6.3 Infeasibility Conditions

The planner must recognize when a goal is mathematically impossible and gracefully reject the request rather than generating a broken schedule. Infeasibility will be triggered under two specific conditions:

- **Deadline Failure:** The minimum baseline time required to complete the critical path of unmastered prerequisites is strictly greater than the total time remaining before the final deadline.
- **Budget Impossibility:** A mandatory, indivisible lesson has a duration that is larger than the entire daily time budget (e.g., a 20-minute lesson for a learner with a strict 15-minute daily limit).

6.4 Output Plan Structure

When a feasible schedule is found, the planner will output a structured state object. This output will include:

- **Daily Schedule:** An ordered mapping of available days to specific instructional and practice activities, keeping strictly within daily budgets.
- **Decision Trace Log:** A machine-readable audit trail attached to the plan that documents exactly which candidate concepts were evaluated, the soft-objective scores they received, and explicit reason codes for any rejected tasks.

7 Evaluation Specification

To rigorously evaluate the system’s performance, specific metrics will be tracked during the interactive testing phase with the simulator. These metrics are categorized into constraint adherence, predictive accuracy, learning outcomes, and system efficiency.

7.1 Constraint Adherence

These metrics will ensure the scheduling algorithm strictly obeys the predefined structural rules:

- **Prerequisite-violation count:** The number of scheduled items whose prerequisites are not yet in the mastered set, K .
- **Daily-budget violation count:** The number of days where the sum of the planned duration ($\Sigma \text{planned_duration}$) strictly exceeds the daily time budget, $B(t)$.
- **Plan feasibility rate:** The percentage of combinations (profile, goal, deadline, budget) where the engine correctly returns either a valid plan or a correct infeasibility report.

7.2 Diagnostic and Predictive Accuracy

These metrics will evaluate how well the mathematical models estimate the learner’s true state:

- **Diagnostic accuracy:** The Mean Absolute Error (MAE) between the diagnostic-estimated mastery and the simulator’s ground-truth mastery, measured at an equal item count.
- **Calibration (Brier score):** The Brier score of the predicted probability of correctness, $P(\text{correct})$, evaluated against actual outcomes on held-out simulated attempts.
- **Mastery-prediction performance:** The Area Under the Curve (AUC) of next-attempt-correctness predictions, including an offline check performed on the ASSISTments logs.

7.3 Learning Outcomes

These metrics will track the effectiveness of the generated study paths for the synthetic profiles:

- **Simulated goal mastery:** The percentage of target concepts reaching the designated mastery threshold by the deadline, evaluated per profile.
- **Review retention:** The simulated recall probability at assessment time for reviewed concepts versus non-reviewed concepts.

7.4 System Efficiency and Explainability

These metrics will validate the practical usability and transparency of the engine:

- **Replanning runtime:** The time in milliseconds (ms) required to produce a valid revised plan following a replanning event.
- **Explanation coverage:** The percentage of scheduled or omitted decisions that are accompanied by a complete, renderable decision trace.

8 Traceability and Repository Setup

To maintain rigorous version control and project management, a dedicated repository will be established. This repository will include a comprehensive **README**, an active issue board for tracking milestones (M1–M8), and a structured decision-log to document architectural changes. Specific test folders will be created to house all initial automated checks, ensuring that test scripts are strictly isolated from core engine logic.

8.1 Initial Acceptance-Test Register

The development milestones will be mapped directly to a suite of automated acceptance tests, providing continuous evidence that the system meets its structural and behavioral requirements.

TC-ID	Milestone	Scenario / Input	Expected Result
TC-01	M3	Schedule a plan on a graph with an unmet mandatory prerequisite.	No lesson for the locked concept is ever scheduled before its prerequisite.
TC-02	M3	Generate a plan under a fixed daily budget.	No day's total scheduled duration exceeds the configured budget.
TC-03	M3	Same learner, same goal, budgets = 5 min vs. 30 min.	The two plans are materially different (different pacing/day count).
TC-04	M2/M3	Two learners with different prior mastery, same goal/budget.	They receive different, individually valid plans.
TC-05	M5/M7	A concept is already mastered above threshold.	It is omitted from new-learning scheduling, with a decision trace citing the mastery evidence.
TC-06	M4	A learner fails an exercise on a low-mastery concept.	Mastery estimate updates downward and a remediation/difficulty response is triggered.
TC-07	M6	Learner misses a scheduled day.	Replanning produces a new valid plan; no completed work is lost or duplicated.
TC-08	M3	Goal cannot fit before the deadline given the budget.	Engine returns an infeasibility report with shortfall and corrective options, not an invalid plan.
TC-09	M5	A high-review-urgency concept and spare daily capacity exist.	The concept receives review priority over lower-urgency new content.
TC-10	M7	Any generated plan (selected and omitted activities).	Every activity's decision can be traced to state, constraints and score components.
TC-11	M1	Load the prerequisite graph.	Graph is confirmed acyclic; cycle injection is rejected at load time.
TC-12	M1	Load curriculum/learner-state schema fixtures.	All required fields are present and typed correctly.